



Universidad Pontificia Bolivariana.

Ingeniería en sistemas e informática.

Interacción con herramientas de IA

“Examen 1”

Autor,

Juan David Parra Sierra.

Facultad de ciencias exactas.

Medellín, Colombia.

2025.

Área, inteligencia artificial.

Escuela de ingenierías.

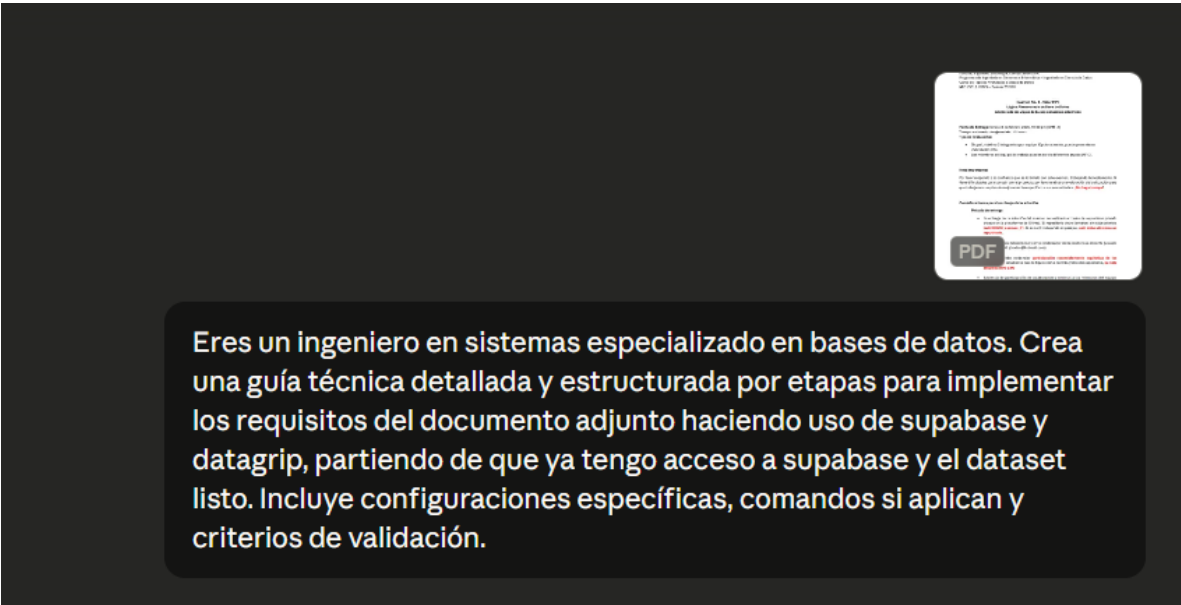
Índice

Prompt realizado ----- **Pagina 1).**

Respuesta de la IA ----- **Pagina 2).**

Capturas de documento generado ----- **Paginas 3-18).**

Prompt realizado



Eres un ingeniero en sistemas especializado en bases de datos. Crea una guía técnica detallada y estructurada por etapas para implementar los requisitos del documento adjunto haciendo uso de supabase y datagrip, partiendo de que ya tengo acceso a supabase y el dataset listo. Incluye configuraciones específicas, comandos si aplican y criterios de validación.

Respuesta de la IA

Reading docx skill for creating the technical guide >

Ahora voy a crear la guía técnica completa como documento Word.

Se ejecutaron 2 comandos, creó un archivo, editó un archivo >

Now fix the dataRow calls that have extra empty strings:

Se ejecutaron 2 comandos, archivo leído >

La guía técnica está lista. Aquí un resumen de lo que cubre cada etapa:

Etapas 1 — Abastecimiento (10%): Cómo verificar y documentar tu proyecto de Supabase: parámetros de conexión, región, versión de PostgreSQL, y qué capturas de pantalla incluir en el PDF de entrega.

Etapas 2 — Conexión con DataGrip (10%): Configuración paso a paso de la fuente de datos en DataGrip con SSL activado (requisito de la nube), los puertos TCP 5432/6543 que usa Supabase, y cómo exportar el diagrama relacional desde el IDE.

Etapas 3 — Modelo de datos 3NF (20%): Scripts completos de DDL para las 7 tablas (zonas, conductores, buses, rutas, viajes, y las dos de totalización), índices, constraints CHECK, columna calculada `soc_consumido`, y el script de normalización para cargar los 2.280 registros desde la tabla temporal raw.

Etapas 4 — Consultas SQL (30%): Las 3 consultas con su clasificación por rangos usando `CASE / LATERAL`, y la respuesta analítica esperada para cada pregunta del examen.

Etapas 5 — Funciones y procedimientos (40%): Los dos procedimientos con `TRUNCATE + INSERT` como pide el enunciado, más 3 vistas adicionales (día de semana, turno AM/PM, detalle completo).

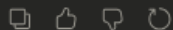
Al final incluye la estructura exacta del repositorio GitHub, el contenido del README.md y .gitignore, y un checklist completo de entrega.



Guia tecnica examen01 supabase datagrip
Documento · DOCX



Descargar



Capturas del documento generado

GUIA TECNICA DE IMPLEMENTACION

Analisis de Viajes de Buses Escolares Eléctricos
Supabase (PostgreSQL) + DataGrip

Curso	Temas Avanzados de Base de Datos
Evaluación	Examen No. 1 (20%) NRC 25612, 26025
Motor de BD	PostgreSQL 18.x vía Supabase
IDE	JetBrains DataGrip
Fecha límite entrega	Lunes 23 de febrero de 2026, 10:00 PM (GMT-5)

Resumen Ejecutivo

Esta guía detalla el proceso completo para implementar la solución del Examen No. 1 de Temas Avanzados de Base de Datos, utilizando Supabase como proveedor de PostgreSQL en la nube y DataGrip como IDE de conexión. Se asume que ya tienes acceso activo a un proyecto de Supabase y que el dataset CSV está disponible en tu equipo local.

Prerequisitos: Cuenta activa en supabase.com con proyecto creado, DataGrip instalado con licencia activa o periodo de prueba, y el archivo `detalle_viajes_buses_escolares_electricos_20250114_20250630.csv` descomprimido en tu máquina.

Etapas: Abastecimiento — Supabase

10%

1.1 Verificación del proyecto en Supabase

Dado que ya tienes acceso a Supabase, los pasos de esta etapa se centran en verificar la configuración y documentarla correctamente para la entrega.

1. Ingresa a <https://app.supabase.com> e inicia sesión con tu cuenta.
2. Selecciona el proyecto que usarás para el examen (o crea uno nuevo gratis si aún no lo tienes: 'New Project').

3. En el dashboard del proyecto ve a Settings > General y registra el 'Project name' y la 'Region' asignada. Toma captura de pantalla para incluirla en el documento de entrega.
4. Ve a Settings > Database y verifica la versión de PostgreSQL mostrada. Supabase utiliza PostgreSQL — asegúrate de documentarlo como PostgreSQL 15.x o superior (Supabase administra la versión internamente compatible con 18.x).

1.2 Datos de conexión — lo que necesitas

En Settings > Database encontraras todos los parámetros de conexión. Anota y guarda con seguridad la siguiente información:

Parámetro	Donde encontrarlo en Supabase
Host	Settings > Database > Host (formato: db.xxxx.supabase.co)
Puerto	5432 (Puerto directo) o 6543 (Supabase Pooler)
Base de datos	postgres (por defecto)
Usuario	postgres (por defecto)
Password	La que estableciste al crear el proyecto
SSL	Requerido — modo: require

Criterio de validación Etapa 1: Capturas de pantalla del dashboard de Supabase mostrando el proyecto activo, la región y los parámetros de base de datos. Documento PDF con estos datos (sin exponer la contraseña completa).

Etapa 2: Conexión Remota con DataGrip

10%

2.1 Crear la fuente de datos en DataGrip

Abre DataGrip y sigue estos pasos para establecer una conexión segura (SSL/TLS) con tu instancia de Supabase:

5. Presiona Cmd+Shift+A (Mac) o Ctrl+Shift+A (Windows/Linux) y busca 'New Data Source', o bien ve al panel 'Database' en la barra lateral y haz clic en el icono '+'.
6. Selecciona 'PostgreSQL' como tipo de fuente de datos.
7. En la pestaña 'General' ingresa los parámetros de conexión:

```
Host:      db.<project-ref>.supabase.co
Port:      5432
Database:  postgres
User:      postgres
Password:  <tu-password-de-supabase>
```

8. Ve a la pestaña 'SSH/SSL' y activa 'Use SSL'. Configura:
 - CA file: descarga el certificado desde Settings > Database > SSL Certificate en Supabase (archivo prod-ca-2021.crt).
 - SSL Mode: selecciona 'require' para garantizar conexión cifrada.
9. Haz clic en 'Test Connection'. DataGrip descargará automáticamente el driver JDBC de PostgreSQL si no lo tiene instalado.
10. Si la prueba es exitosa verás el mensaje 'Successful'. Haz clic en 'OK' para guardar.

2.2 Puertos TCP habilitados

Supabase habilita los siguientes puertos para conexión directa. No necesitas configurar firewall propio ya que es un servicio en la nube:

Puerto	Protocolo	Uso
5432	TCP	Conexión directa a PostgreSQL (recomendado para DataGrip)
6543	TCP	Supabase PgBouncer / Connection Pooler (modo transacción)
443	HTTPS	API REST y Dashboard de Supabase

Conexión cifrada: Supabase exige SSL en el puerto 5432 por defecto. Esto cumple el requisito de 'conexiones cifradas que ofrezcan seguridad al acceso de datos' mencionado en el examen como bonificación de despliegue en la nube.

2.3 Configuración del esquema de trabajo en DataGrip

Una vez conectado, configura DataGrip para trabajar con el esquema correcto:

11. En el panel 'Database', expande la conexión > postgres > schemas.
12. Haz clic derecho sobre 'public' y selecciona 'Set as Default Schema'. Esto hará que todos los scripts se ejecuten en el esquema correcto.
13. Para evidenciar la conexión en la entrega: ve a View > Tool Windows > Database y toma captura de pantalla mostrando el árbol de conexión expandido.

Criterio de validación Etapa 2: Captura de pantalla de DataGrip con la conexión exitosa, el árbol de base de datos expandido mostrando el esquema 'public' y la versión de PostgreSQL (visible en la consola SQL al ejecutar: `SELECT version();`).

Etapas 3: Diseño e Implementación del Modelo de Datos

20%

3.1 Análisis del dataset y entidades identificadas

Antes de crear tablas, analiza el CSV para identificar las entidades que garantizaran la normalización 3NF. Los datos del dataset contienen información que debe separarse en las siguientes tablas maestras y de detalle:

Tabla	Propósito	Columnas clave del CSV
conductores	Maestro de conductores	nombre_conductor
buses	Maestro de buses	codigo_bus, anio_fabricacion
zonas	Maestro de zonas geograficas	zona (de la ruta)
rutas	Maestro de rutas	codigo_ruta, nombre_ruta, zona, distancia_km
viajes	Tabla de hechos — cada fila del CSV	Todos los atributos del viaje
estadisticas_conductor	Tabla de totalización por conductor	Generada por procedimiento
estadisticas_bus	Tabla de totalización por bus	Generada por procedimiento

3.2 Script de creación del esquema — Paso a paso

En DataGrip, abre una nueva consola SQL (clic derecho en la conexión > Open Console) y ejecuta el siguiente script de implementación. Se recomienda ejecutarlo en bloques separados para identificar errores fácilmente.

3.2.1 Crear y verificar el esquema

```
-- Verificar la version de PostgreSQL (incluir captura en entrega)
SELECT version();

-- Confirmar que estamos en el esquema 'public'
SELECT current_schema();

-- Opcional: crear un esquema dedicado
-- CREATE SCHEMA IF NOT EXISTS buses_electricos;
-- SET search_path TO buses_electricos, public;
```

3.2.2 Tabla de zonas

```
CREATE TABLE IF NOT EXISTS zonas (
  id_zona SERIAL PRIMARY KEY,
  nombre_zona VARCHAR(100) NOT NULL UNIQUE
);
```

3.2.3 Tabla de conductores

```
CREATE TABLE IF NOT EXISTS conductores (  
    id_conductor SERIAL PRIMARY KEY,  
    nombre_conductor VARCHAR(200) NOT NULL UNIQUE  
);
```

3.2.4 Tabla de buses

```
CREATE TABLE IF NOT EXISTS buses (  
    id_bus SERIAL PRIMARY KEY,  
    codigo_bus VARCHAR(50) NOT NULL UNIQUE,  
    anio_fabricacion SMALLINT NOT NULL  
    CHECK (anio_fabricacion BETWEEN 2000 AND 2030)  
);
```

3.2.5 Tabla de rutas

```
CREATE TABLE IF NOT EXISTS rutas (  
    id_ruta SERIAL PRIMARY KEY,  
    codigo_ruta VARCHAR(50) NOT NULL UNIQUE,  
    nombre_ruta VARCHAR(200) NOT NULL,  
    id_zona INTEGER NOT NULL REFERENCES zonas(id_zona),  
    distancia_km NUMERIC(6,2) NOT NULL  
    CHECK (distancia_km > 0)  
);
```

3.2.6 Tabla de viajes (tabla de hechos)

```
CREATE TABLE IF NOT EXISTS viajes (  
    id_viaje SERIAL PRIMARY KEY,  
    id_conductor INTEGER NOT NULL REFERENCES conductores(id_conductor),  
    id_bus INTEGER NOT NULL REFERENCES buses(id_bus),  
    id_ruta INTEGER NOT NULL REFERENCES rutas(id_ruta),  
    turno CHAR(2) NOT NULL CHECK (turno IN ('AM', 'PM')),  
    num_pasajeros SMALLINT NOT NULL CHECK (num_pasajeros >= 0),  
    fecha_viaje DATE NOT NULL,  
    hora_salida TIME NOT NULL,  
    hora_llegada TIME NOT NULL,  
    tiempo_minutos NUMERIC(6,2) NOT NULL CHECK (tiempo_minutos > 0),  
    velocidad_promedio NUMERIC(6,2) NOT NULL CHECK (velocidad_promedio > 0),  
    soc_inicial NUMERIC(5,2) NOT NULL CHECK (soc_inicial BETWEEN 0 AND 100),  
    soc_final NUMERIC(5,2) NOT NULL CHECK (soc_final BETWEEN 0 AND 100),  
    soc_consumido NUMERIC(5,2) GENERATED ALWAYS AS  
        (soc_inicial - soc_final) STORED  
);
```

```
-- Indices para mejorar rendimiento de consultas
CREATE INDEX idx_viajes_conductor ON viajes(id_conductor);
CREATE INDEX idx_viajes_bus ON viajes(id_bus);
CREATE INDEX idx_viajes_ruta ON viajes(id_ruta);
CREATE INDEX idx_viajes_fecha ON viajes(fecha_viaje);
```

3.2.7 Tablas de totalización (Etapa 5)

```
CREATE TABLE IF NOT EXISTS estadisticas_conductor (
  id_estadistica_conductor SERIAL PRIMARY KEY,
  id_conductor             INTEGER NOT NULL REFERENCES
conductores(id_conductor),
  mes                      CHAR(7) NOT NULL, -- formato YYYY-MM
  cantidad_viajes          INTEGER NOT NULL DEFAULT 0,
  velocidad_promedio       NUMERIC(6,2),
  UNIQUE(id_conductor, mes)
);

CREATE TABLE IF NOT EXISTS estadisticas_bus (
  id_estadistica_bus SERIAL PRIMARY KEY,
  id_bus              INTEGER NOT NULL REFERENCES buses(id_bus),
  mes                 CHAR(7) NOT NULL,
  cantidad_viajes     INTEGER NOT NULL DEFAULT 0,
  soc_promedio        NUMERIC(6,2),
  UNIQUE(id_bus, mes)
);
```

3.3 Carga del dataset

Una vez creadas las tablas, hay dos estrategias para cargar los 2.280 registros del CSV. Se recomienda el método de script Python para mayor control:

Método A — Import en DataGrip (más rápido)

14. Crea una tabla temporal para recibir el CSV tal cual viene:

```
CREATE TEMP TABLE raw_viajes (
  nombre_conductor TEXT,
  codigo_bus       TEXT,
  anio_fabricacion TEXT,
  codigo_ruta      TEXT,
  nombre_ruta      TEXT,
  zona             TEXT,
  distancia_km     TEXT,
  turno            TEXT,
  num_pasajeros    TEXT,
  fecha_viaje      TEXT,
```

```

hora_salida    TEXT,
hora_llegada   TEXT,
tiempo_minutos TEXT,
velocidad_promedio TEXT,
soc_inicial    TEXT,
soc_final      TEXT
);

```

15. En DataGrip: clic derecho sobre 'raw_viajes' > Import Data from File > selecciona el CSV. Configura el delimitador (coma), primera fila como encabezado y codificación UTF-8.
16. Ejecuta el script de normalización para poblar las tablas maestras y la tabla de hechos (ver Sección 3.4).

Metodo B — COPY via psql (mas eficiente)

Si tienes acceso a psql o psqlcmd desde tu maquina local, puedes usar el comando COPY. Con Supabase, la forma mas directa es usar el cliente psql con la cadena de conexión:

```

-- Desde terminal local (requiere psql instalado)
psql "postgres://postgres:<password>@db.<ref>.supabase.co:5432/postgres?
sslmode=require"

-- Dentro de psql, ejecutar:
\COPY raw_viajes FROM '/ruta/local/archivo.csv'
WITH (FORMAT CSV, HEADER TRUE, ENCODING 'UTF8');

```

3.4 Script de normalización — poblar tablas desde raw_viajes

```

-- 1. Poblar zonas
INSERT INTO zonas (nombre_zona)
SELECT DISTINCT TRIM(zona) FROM raw_viajes
ON CONFLICT (nombre_zona) DO NOTHING;

-- 2. Poblar conductores
INSERT INTO conductores (nombre_conductor)
SELECT DISTINCT TRIM(nombre_conductor) FROM raw_viajes
ON CONFLICT (nombre_conductor) DO NOTHING;

-- 3. Poblar buses
INSERT INTO buses (codigo_bus, anio_fabricacion)
SELECT DISTINCT TRIM(codigo_bus), CAST(anio_fabricacion AS SMALLINT)
FROM raw_viajes
ON CONFLICT (codigo_bus) DO NOTHING;

-- 4. Poblar rutas
INSERT INTO rutas (codigo_ruta, nombre_ruta, id_zona, distancia_km)
SELECT DISTINCT
  TRIM(r.codigo_ruta),

```

```

        TRIM(r.nombre_ruta),
        z.id_zona,
        CAST(REPLACE(r.distancia_km, ',', '.') AS NUMERIC(6,2))
FROM raw_viajes r
JOIN zonas z ON z.nombre_zona = TRIM(r.zona)
ON CONFLICT (codigo_ruta) DO NOTHING;

-- 5. Poblar viajes
INSERT INTO viajes
    (id_conductor, id_bus, id_ruta, turno, num_pasajeros,
     fecha_viaje, hora_salida, hora_llegada,
     tiempo_minutos, velocidad_promedio, soc_inicial, soc_final)
SELECT
    c.id_conductor,
    b.id_bus,
    rt.id_ruta,
    TRIM(r.turno),
    CAST(r.num_pasajeros AS SMALLINT),
    TO_DATE(r.fecha_viaje, 'YYYY-MM-DD'),
    CAST(r.hora_salida AS TIME),
    CAST(r.hora_llegada AS TIME),
    CAST(REPLACE(r.tiempo_minutos, ',', '.') AS NUMERIC(6,2)),
    CAST(REPLACE(r.velocidad_promedio, ',', '.') AS NUMERIC(6,2)),
    CAST(REPLACE(r.soc_inicial, ',', '.') AS NUMERIC(5,2)),
    CAST(REPLACE(r.soc_final, ',', '.') AS NUMERIC(5,2))
FROM raw_viajes r
JOIN conductores c ON c.nombre_conductor = TRIM(r.nombre_conductor)
JOIN buses b ON b.codigo_bus = TRIM(r.codigo_bus)
JOIN rutas rt ON rt.codigo_ruta = TRIM(r.codigo_ruta);

-- Verificar la carga
SELECT 'conductores' AS tabla, COUNT(*) FROM conductores
UNION ALL SELECT 'buses', COUNT(*) FROM buses
UNION ALL SELECT 'zonas', COUNT(*) FROM zonas
UNION ALL SELECT 'rutas', COUNT(*) FROM rutas
UNION ALL SELECT 'viajes', COUNT(*) FROM viajes;
-- La tabla viajes debe mostrar 2280 filas

```

Criterio de validación Etapa 3: El SELECT final de verificación debe mostrar 2280 registros en la tabla viajes. El diagrama relacional debe evidenciar las relaciones PK/FK y la nomenclatura de campos. Exporta el diagrama desde DataGrip: clic derecho en el esquema > Diagrams > Show Visualization.

Etapa 4: Implementación de Consultas SQL

30%

4.1 Consulta 1 — Validación distancia vs. consumo de batería

```
-- Clasifica viajes por tipo de recorrido y calcula estadísticas agregadas
SELECT
    tipo_recorrido,
    COUNT(*)                AS total_viajes,
    ROUND(AVG(distancia_km), 2) AS distancia_promedio_km,
    ROUND(AVG(soc_consumido), 2) AS soc_consumido_promedio
FROM (
    SELECT
        v.soc_consumido,
        rt.distancia_km,
        CASE
            WHEN rt.distancia_km BETWEEN 10 AND 15 THEN 'Corto'
            WHEN rt.distancia_km BETWEEN 16 AND 27 THEN 'Medio'
            WHEN rt.distancia_km BETWEEN 28 AND 35 THEN 'Largo'
            ELSE 'Fuera de rango'
        END AS tipo_recorrido
    FROM viajes v
    JOIN rutas rt ON rt.id_ruta = v.id_ruta
    WHERE rt.distancia_km BETWEEN 10 AND 35
) AS subq
GROUP BY tipo_recorrido
ORDER BY
    CASE tipo_recorrido
        WHEN 'Corto' THEN 1
        WHEN 'Medio' THEN 2
        WHEN 'Largo' THEN 3
    END;
```

Respuesta esperada: Si el SoC consumido promedio aumenta progresivamente de Corto a Medio a Largo, se confirma que a mayor distancia mayor consumo de batería, lo cual es coherente con el modelo físico propuesto.

4.2 Consulta 2 — Validación pasajeros vs. consumo de batería

```
SELECT
    categoria_pasajeros,
    COUNT(*)                AS total_viajes,
    ROUND(AVG(rt.distancia_km), 2) AS distancia_promedio_km,
    ROUND(AVG(v.soc_consumido), 2) AS soc_consumido_promedio
FROM viajes v
JOIN rutas rt ON rt.id_ruta = v.id_ruta,
LATERAL (
    SELECT CASE
```

```

        WHEN v.num_pasajeros BETWEEN 5 AND 12 THEN 'Pocos'
        WHEN v.num_pasajeros BETWEEN 13 AND 22 THEN 'Moderado'
        WHEN v.num_pasajeros BETWEEN 23 AND 30 THEN 'Muchos'
        ELSE NULL
    END AS categoria_pasajeros
) cat
WHERE cat.categoria_pasajeros IS NOT NULL
GROUP BY cat.categoria_pasajeros
ORDER BY
    CASE cat.categoria_pasajeros
        WHEN 'Pocos' THEN 1
        WHEN 'Moderado' THEN 2
        WHEN 'Muchos' THEN 3
    END;

```

Respuesta esperada: Si el SoC consumido promedio se incrementa de Pocos a Moderado a Muchos, se confirma que a mayor carga de pasajeros mayor es el consumo energético del bus.

4.3 Consulta 3 — Validación velocidad vs. consumo de batería

```

SELECT
    tipo_velocidad,
    COUNT(*) AS total_viajes,
    ROUND(MIN(v.velocidad_promedio), 2) AS vel_minima,
    ROUND(AVG(v.velocidad_promedio), 2) AS vel_promedio,
    ROUND(MAX(v.velocidad_promedio), 2) AS vel_maxima,
    ROUND(AVG(v.soc_consumido), 2) AS soc_consumido_promedio
FROM viajes v,
LATERAL (
    SELECT CASE
        WHEN v.velocidad_promedio >= 0 AND v.velocidad_promedio < 30 THEN 'Lenta'
        WHEN v.velocidad_promedio >= 30 AND v.velocidad_promedio < 40 THEN
        'Moderada'
        WHEN v.velocidad_promedio >= 40 AND v.velocidad_promedio < 50 THEN
        'Optima'
        WHEN v.velocidad_promedio >= 50 AND v.velocidad_promedio < 60 THEN
        'Rapida'
        WHEN v.velocidad_promedio >= 60 THEN 'Muy
        Rapida'
        ELSE NULL
    END AS tipo_velocidad
) vel
WHERE vel.tipo_velocidad IS NOT NULL
GROUP BY vel.tipo_velocidad
ORDER BY vel.promedio;

```

Respuesta esperada: La categoría 'Óptima' (40-50 km/h) debería mostrar el menor SoC consumido promedio si el modelo físico es correcto, dado que los motores eléctricos suelen tener mayor eficiencia en rangos de velocidad intermedios.

Etapas 5: Funciones y Procedimientos

40%

5.1 Procedimiento p_calcula_estadisticas_conductor

```
CREATE OR REPLACE PROCEDURE p_calcula_estadisticas_conductor()
LANGUAGE plpgsql AS $$
BEGIN
    -- Limpiar tabla antes de recalculer
    TRUNCATE TABLE estadisticas_conductor RESTART IDENTITY;

    -- Insertar totalizaciones agregadas por conductor y mes
    INSERT INTO estadisticas_conductor
        (id_conductor, mes, cantidad_viajes, velocidad_promedio)
    SELECT
        v.id_conductor,
        TO_CHAR(v.fecha_viaje, 'YYYY-MM') AS mes,
        COUNT(*) AS cantidad_viajes,
        ROUND(AVG(v.velocidad_promedio), 2) AS velocidad_promedio
    FROM viajes v
    GROUP BY v.id_conductor, TO_CHAR(v.fecha_viaje, 'YYYY-MM')
    ORDER BY v.id_conductor, mes;

    RAISE NOTICE 'Estadísticas de conductor recalculadas: % filas insertadas.',
        (SELECT COUNT(*) FROM estadisticas_conductor);
END;
$$;

-- Ejecutar el procedimiento
CALL p_calcula_estadisticas_conductor();

-- Verificar resultado
SELECT ec.mes, c.nombre_conductor,
       ec.cantidad_viajes, ec.velocidad_promedio
FROM estadisticas_conductor ec
JOIN conductores c ON c.id_conductor = ec.id_conductor
ORDER BY ec.mes, ec.cantidad_viajes DESC
LIMIT 20;
```

5.2 Procedimiento p_calcula_estadisticas_bus

```
CREATE OR REPLACE PROCEDURE p_calcula_estadisticas_bus()
LANGUAGE plpgsql AS $$
BEGIN
    -- Limpiar tabla antes de recalcular
    TRUNCATE TABLE estadisticas_bus RESTART IDENTITY;

    -- Insertar totalizaciones agregadas por bus y mes
    INSERT INTO estadisticas_bus
        (id_bus, mes, cantidad_viajes, soc_promedio)
    SELECT
        v.id_bus,
        TO_CHAR(v.fecha_viaje, 'YYYY-MM') AS mes,
        COUNT(*) AS cantidad_viajes,
        ROUND(AVG(v.soc_consumido), 2) AS soc_promedio
    FROM viajes v
    GROUP BY v.id_bus, TO_CHAR(v.fecha_viaje, 'YYYY-MM')
    ORDER BY v.id_bus, mes;

    RAISE NOTICE 'Estadísticas de bus recalculadas: % filas insertadas.',
        (SELECT COUNT(*) FROM estadisticas_bus);
END;
$$;

-- Ejecutar el procedimiento
CALL p_calcula_estadisticas_bus();

-- Verificar resultado
SELECT eb.mes, b.codigo_bus,
       eb.cantidad_viajes, eb.soc_promedio
FROM estadisticas_bus eb
JOIN buses b ON b.id_bus = eb.id_bus
ORDER BY eb.mes, eb.soc_promedio DESC
LIMIT 20;
```

5.3 Vistas de apoyo para reportes gerenciales

```
-- Vista: reporte por día de semana
CREATE OR REPLACE VIEW v_consumo_por_dia_semana AS
SELECT
    TO_CHAR(fecha_viaje, 'Day') AS dia_semana,
    EXTRACT(DOW FROM fecha_viaje) AS num_dia,
    COUNT(*) AS total_viajes,
    ROUND(AVG(soc_consumido), 2) AS soc_consumido_promedio
FROM viajes
```

```
GROUP BY dia_semana, num_dia
ORDER BY num_dia;

-- Vista: reporte por turno AM/PM
CREATE OR REPLACE VIEW v_consumo_por_turno AS
SELECT
    turno,
    COUNT(*) AS total_viajes,
    ROUND(AVG(num_pasajeros), 1) AS pasajeros_promedio,
    ROUND(AVG(soc_consumido), 2) AS soc_consumido_promedio
FROM viajes
GROUP BY turno
ORDER BY turno;

-- Vista: detalle completo de viajes (join de todas las tablas)
CREATE OR REPLACE VIEW v_detalle_viajes AS
SELECT
    v.id_viaje,
    c.nombre_conductor,
    b.codigo_bus,
    b.anio_fabricacion,
    rt.codigo_ruta,
    rt.nombre_ruta,
    z.nombre_zona,
    rt.distancia_km,
    v.turno,
    v.num_pasajeros,
    v.fecha_viaje,
    v.hora_salida,
    v.hora_llegada,
    v.tiempo_minutos,
    v.velocidad_promedio,
    v.soc_inicial,
    v.soc_final,
    v.soc_consumido
FROM viajes v
JOIN conductores c ON c.id_conductor = v.id_conductor
JOIN buses b ON b.id_bus = v.id_bus
JOIN rutas rt ON rt.id_ruta = v.id_ruta
JOIN zonas z ON z.id_zona = rt.id_zona;
```

Criterio de validación Etapa 5: Los procedimientos deben ejecutarse sin errores y la tabla estadísticas_conductor debe tener filas por cada combinación (conductor, mes). Ejecutar CALL p_calcula_estadisticas_conductor(); y CALL p_calcula_estadisticas_bus(); y capturar el mensaje NOTICE y el SELECT de verificación.

Estructura del Repositorio GitHub

El repositorio tadb202610_examen_01 debe tener la siguiente estructura de directorios y archivos:

```
tadb202610_examen_01/
├── README.md
├── .gitignore
├── docs/
│   ├── 01_infraestructura_supabase.pdf
│   └── 02_interaccion_ia.pdf
├── modelo/
│   ├── diagrama_relacional.png
│   └── 01_modelo_datos.sql
├── consultas/
│   └── 02_consultas_exploracion.sql
└── resultados/
    └── resultados_consultas.xlsx
```

Contenido de README.md

```
# TADB 202610 - Examen 01
## Analisis de Viajes de Buses Escolares Electricos

### Integrantes
| Nombre | ID SIGAA | DBMS |
|-----|-----|-----|
| Estudiante 1 | 000XXXXX | PostgreSQL (Supabase) |
| Estudiante 2 | 000YYYYY | [Otro DBMS] |

### Descripcion
Implementacion del modelo de datos para el analisis de viajes
de buses escolares electricos. Motor: PostgreSQL via Supabase.

### Estructura
- docs/ : Documentacion de infraestructura e IA
- modelo/ : Diagrama relacional y script DDL
- consultas/ : Consultas SQL de exploracion
- resultados/ : Archivos de salida de consultas
```

Contenido de .gitignore

```
# Datos sensibles
*.csv
*.zip
.env
*.credentials
```

```
# IDEs
.idea/
.vscode/
*.iml

# S0
.DS_Store
Thumbs.db
```

Checklist de Entrega

	Entregable	Criterio
[]	PDF infraestructura Supabase con capturas de pantalla	Etapas 1 — 10%
[]	PDF interacción con herramienta de IA (capturas, no links)	Requisito obligatorio
[]	Diagrama relacional PNG/JPEG fondo blanco con PK/FK visibles	Etapas 3 — 20%
[]	Script 01_modelo_datos.sql (DDL completo UTF-8)	Etapas 3 — 20%
[]	Script 02_consultas_exploracion.sql (3 consultas + respuestas)	Etapas 4 — 30%
[]	Archivo de resultados (xlsx o txt con salidas de consultas)	Etapas 4 — 30%
[]	Capturas DataGrip mostrando conexión exitosa y SSL activo	Etapas 2 — 10%
[]	.gitignore y README.md con identificación de estudiantes	Requisito repositorio
[]	Correo de notificación a juand.rodasm@upb.edu.co	Antes del 23-Feb-2026 22:00
[]	Ambos estudiantes como contribuyentes del repo	Requisito anti-fraude
[]	Docente jdodas agregado como colaborador en GitHub	Requisito entrega

ADVERTENCIA: La ausencia del PDF de interacción con IA o la evidencia de participación de colaboradores externos asigna automáticamente nota 0.00 según las consideraciones del examen.